

Bases de Datos Relacionales

CRUD, diseño e integridad en SQL puro

Píldora técnica · 7 de octubre · 45 minutos

Rita Pereira

IA School Barcelona · P1

¿Dónde vive todo lo que usamos a diario?

¿Alguna vez habéis buscado algo en Instagram, o mirado vuestro saldo en la app del banco?

Todo eso vive guardado en algún sitio, ordenado, para poder encontrarse en un instante.

Hoy vamos a abrir esa caja.



¿Por qué necesitamos organizar la información?



Una biblioteca gigante

Como una biblioteca con miles de libros: sin orden, encontrar uno tardaría horas.



Con estructura, todo cuadra

Tablas, filas y columnas permiten que un ordenador encuentre exactamente lo que le pides, en milisegundos.



Sin errores ni duplicados

Evita que el mismo cliente aparezca tres veces con tres direcciones distintas.

Tipos de bases de datos

Relacionales (SQL)

- Datos organizados en tablas conectadas entre sí.
- MySQL, PostgreSQL, SQL Server.
- La base de casi cualquier sistema empresarial: bancos, tiendas online, hospitales.

No relacionales (NoSQL)

- Datos más flexibles, sin una estructura fija de tablas.
- MongoDB, Firebase.
- Útiles para datos muy variables o de gran volumen.

Hoy nos centramos en las relacionales.

02

Bases de Datos Relacionales, explicadas de verdad

(como si tuvierais 5 años)

Imagina que tienes muchos juguetes...

Una base de datos relacional es como tener varios cajones especiales, cada uno para un tipo de cosa.



Tablas = cajones

Un cajón para "Niños", otro para "Juguetes". Cada cajón guarda solo un tipo de información.



Filas = tarjetas

Dentro del cajón "Juguetes", cada tarjeta es un juguete distinto: el osito, el coche rojo, el balón.



Columnas = etiquetas

En la tarjeta del osito escribes: nombre, color, tamaño. Todas las tarjetas del cajón tienen las mismas etiquetas.



Relaciones = un número secreto

Si Juan es el "1" y el osito dice "dueño: 1", sabemos que ese osito es de Juan. Ese número conecta los cajones.

De la analogía al concepto técnico

Analogía	Concepto técnico	Definición corta
Cajón	Tabla	Colección de datos del mismo tipo (ej. Clientes, Pedidos)
Tarjeta	Fila / Registro	Un elemento individual dentro de la tabla
Etiqueta de la tarjeta	Columna / Atributo	Una característica del registro (nombre, edad, precio)
Número único de cada tarjeta	Clave Primaria (Primary Key)	Identificador único de cada fila en su tabla
Número que apunta a otro cajón	Clave Foránea (Foreign Key)	Campo que conecta una fila con una fila de otra tabla
Que todo cuadre, sin contradicciones	Integridad de datos	Garantía de que las relaciones entre tablas son coherentes

Conectando dos tablas con una llave

NIÑOS

id	nombre
1	Juan
2	Ana

JUGUETES

id	nombre	dueño_id
1	Osito peluche	1
2	Coche rojo	2
3	Balón	1



"dueño_id" en JUGUETES es la clave foránea que apunta al "id" (clave primaria) de NIÑOS.

Así el ordenador sabe que el osito y el balón son de Juan (id = 1), y el coche rojo es de Ana (id = 2).

IN ENGLISH

The real-world vocabulary

In the real world, you'll hear these terms in English — table, row, column, primary key, foreign key. Even if you work in Spanish, the code and the documentation will almost always use these English words, so it's worth getting comfortable with them early.

table	row	column	primary key	foreign key

mesa · fila · columna · clave primaria · clave foránea

03

Trabajando con SQL puro

Cómo se crea, consulta, modifica y protege la información, sin salir del lenguaje SQL

CRUD, pero en SQL puro

Las mismas cuatro operaciones de siempre, escritas directamente en el lenguaje de la base de datos.



C — Create

`INSERT`

Añade una fila nueva a una tabla.



R — Read

`SELECT`

Consulta una o varias filas.



U — Update

`UPDATE`

Modifica los valores de filas existentes.



D — Delete

`DELETE`

Elimina filas de una tabla.

Create y Read en SQL

INSERT — añadir un libro

```
INSERT INTO libros
  (título, autor, año)
VALUES
  ('Cien años de soledad',
   'García Márquez', 1967);
```

SELECT — consultar libros

```
SELECT título, autor, año
FROM libros
WHERE año > 1960
ORDER BY año;
```

Update y Delete en SQL

UPDATE — cambiar un dato

```
UPDATE libros
SET título = 'Cien años de
  soledad (Ed. revisada)'
WHERE id = 1;
```

DELETE — borrar una fila

```
DELETE FROM libros
WHERE id = 1;
```

⚠️ *UPDATE y DELETE sin WHERE afectan a toda la tabla. Siempre se prueba antes el WHERE con un SELECT.*

Modelado de datos: la normalización

Diseñar bien una tabla es decidir qué información va en cada cajón, para no repetir datos ni perder coherencia.

1FN Cada celda guarda un solo valor (nada de "Juan, Ana" en una misma casilla).

2FN Cada columna depende de toda la clave primaria, no solo de una parte.

3FN Ninguna columna depende de otra columna que no sea la clave (sin dependencias transitivas).

¿Por qué molestarse? Evita que, al actualizar un dato, tengas que cambiarlo en diez sitios distintos y te olvides de alguno.

2FN en un ejemplo: pedidos y productos

Un pedido puede tener varios productos, y un producto puede estar en varios pedidos: por eso hace falta una tabla "puente" con clave compuesta (pedido_id + producto_id).

✗ Antes — rompe 2FN

pedido_id	producto_id	nombre_producto	cantidad
101	5	Cuaderno A5	2
101	8	Bolígrafo azul	1
102	5	Cuaderno A5	3

"nombre_producto" solo depende de producto_id, no de la clave completa (pedido_id + producto_id) → se repite y si cambia el nombre, hay que actualizarlo en varias filas.

✓ Después — cumple 2FN

pedidos_productos

pedido_id	producto_id	cantidad
101	5	2
101	8	1
102	5	3

productos

id	nombre_producto
5	Cuaderno A5
8	Bolígrafo azul

3FN en un ejemplo: empleados y departamentos

Aquí la clave primaria es solo "id" (no hay clave compuesta). El problema es otra columna que depende de "departamento_id" en vez de depender del empleado.

✗ Antes — rompe 3FN

id	nombre	departamento_id	nombre_departamento
1	Juan	10	Ventas
2	Ana	10	Ventas
3	Marta	20	Soporte

"nombre_departamento" no depende del empleado (la clave, "id"), depende de "departamento_id" → dependencia transitiva. Si "Ventas" cambia de nombre, hay que tocar varias filas.

✓ Después — cumple 3FN

empleados

id	nombre	departamento_id
1	Juan	10
2	Ana	10
3	Marta	20

departamentos

id	nombre_departamento
10	Ventas
20	Soporte

Integridad referencial: las reglas que no se rompen

La normalización evita que un dato se repita. Pero aunque las tablas estén bien separadas, nada impide todavía meter un valor imposible o una referencia inventada. Para eso existen las restricciones de integridad: reglas que la base de datos aplica siempre, sin excepción.



NOT NULL

Un campo obligatorio nunca puede quedar vacío (ej. todo cliente necesita un nombre).



UNIQUE

Un valor no se puede repetir en esa columna (ej. dos usuarios no pueden tener el mismo email).



FOREIGN KEY

Una fila no puede apuntar a algo que no existe (ej. un pedido no puede tener un cliente_id inventado).



CHECK

El valor debe cumplir una condición (ej. el precio nunca puede ser negativo).

Transacciones ACID

La normalización y la integridad garantizan que el diseño sea correcto, pero no evitan que algo falle a mitad de una operación (se cae el sistema, hay un error). Para eso existen las transacciones: agrupan varios pasos SQL en uno solo, con cuatro garantías.

Ejemplo: transferir 100 € de la cuenta A a la cuenta B son dos UPDATE seguidos, que deben ocurrir juntos o ninguno.



A — Atomicidad

Todo o nada. Si falla el paso 2, el paso 1 se deshace: nadie pierde dinero por el camino.



C — Consistencia

La base de datos pasa de un estado válido a otro: el dinero total no cambia, solo se mueve.



I — Aislamiento

Dos transacciones a la vez no se pisan: es como si cada una ocurriera sola.



D — Durabilidad

Una vez confirmada, la transacción sobrevive aunque el sistema se caiga justo después.

IN ENGLISH

ACID: the words behind the guarantee

This acronym is always used in English, even in Spanish-speaking teams: Atomicity, Consistency, Isolation, Durability. You'll see it in almost every database course and every technical interview.

Atomicity	Consistency	Isolation	Durability
Atomicidad	*Consistencia*	*Aislamiento*	*Durabilidad*

Consultas SQL: uniendo tablas de verdad

Ya sabemos diseñar las tablas (normalización), protegerlas (integridad, ACID) y leerlas una a una (SELECT). Ahora volvemos al CRUD, al "Read", pero para la pregunta que más se repite en el día a día: recuperar información combinando varias tablas a la vez, donde el modelo relacional demuestra su fuerza.

JOIN — combinar dos tablas

```
SELECT niños.nombre, juguetes.nombre
FROM niños
JOIN juguetes
  ON juguetes.dueño_id = niños.id;
```

GROUP BY — agregar resultados

```
SELECT niños.nombre, COUNT(*) AS total
FROM niños
JOIN juguetes
  ON juguetes.dueño_id = niños.id
GROUP BY niños.nombre
HAVING COUNT(*) > 1;
```

JOIN conecta filas de dos tablas por su llave; GROUP BY + HAVING agrupa y filtra sobre esos grupos: aquí, los niños con más de un juguete.

Tipos de JOIN: ¿cuál uso?

Todos combinan dos tablas; lo que cambia es qué filas se quedan fuera cuando no hay pareja al otro lado.

INNER JOIN



Tabla A Tabla B

Solo las filas que coinciden en ambas tablas.

Cuando solo te interesa lo que sí tiene relación (ej. pedidos que sí tienen cliente asignado).

LEFT JOIN



Tabla A Tabla B

Todas las filas de la tabla izquierda, tengan o no pareja a la derecha.

Para ver todo de tu tabla principal, aunque le falte información relacionada (ej. todos los clientes, incluso sin pedidos).

RIGHT JOIN



Tabla A Tabla B

Todas las filas de la tabla derecha, tengan o no pareja a la izquierda.

Lo mismo que LEFT JOIN visto desde el otro lado; en la práctica casi siempre se reescribe como LEFT JOIN cambiando el orden de las tablas.

FULL OUTER JOIN



Tabla A Tabla B

Todas las filas de ambas tablas, coincidan o no.

Para detectar a la vez huecos en los dos sentidos (ej. clientes sin pedidos y pedidos sin cliente).

La diferencia, en una consulta real

Partimos de un niño sin juguetes todavía (Marta, id = 3) para ver qué cambia.

INNER JOIN — descarta a Marta

```
SELECT niños.nombre,
       juguetes.nombre
FROM niños
JOIN juguetes
  ON juguetes.dueño_id = niños.id;

-- Resultado: Juan, Ana
-- (Marta no aparece: no
--  tiene juguetes)
```

LEFT JOIN — conserva a Marta

```
SELECT niños.nombre,
       juguetes.nombre
FROM niños
LEFT JOIN juguetes
  ON juguetes.dueño_id = niños.id;

-- Resultado: Juan, Ana, Marta
-- (Marta aparece con
--  juguete = NULL)
```

Regla práctica: si un JOIN normal te hace "perder" filas que esperabas ver, casi siempre la solución es un LEFT JOIN.

Índices: el atajo para encontrar datos rápido

Los JOIN y las consultas que acabamos de ver funcionan bien con pocas filas, pero una tabla real puede tener millones. ¿Cómo evitamos que esas mismas consultas se vuelvan lentas al crecer la tabla? Con índices.



Un índice es como el índice de un libro: en vez de leer página por página para encontrar un capítulo, vas directo a la página correcta.



Sin índice

La base de datos revisa fila por fila hasta encontrar lo que buscas ("full table scan"). Lento en tablas grandes.



Con índice

La base de datos salta directo a las filas relevantes, usando una estructura ordenada (normalmente un árbol B).

El precio a pagar: cada índice ocupa espacio y ralentiza un poco las escrituras (INSERT/UPDATE), así que se indexa con criterio, no todo.

Índices en código: la misma consulta, dos escenarios

Partimos de una tabla "libros" con miles de filas, sin ningún índice creado todavía.

Sin índice — escaneo completo

```
SELECT *
FROM libros
WHERE autor = 'Isabel Allende';

-- La base de datos revisa
-- fila por fila, una a una,
-- hasta encontrar todas las
-- coincidencias.
-- Con miles de filas, se nota.
```

Con índice — salto directo

```
CREATE INDEX idx_autor
ON libros (autor);

SELECT *
FROM libros
WHERE autor = 'Isabel Allende';

-- Misma consulta, mismo
-- resultado. Pero ahora la
-- base de datos usa el índice
-- de "autor" y salta directo
-- a las filas que coinciden.
```

El CREATE INDEX solo se ejecuta una vez. A partir de ahí, todas las consultas que filtren por "autor" se benefician del atajo.

Lo que nos llevamos hoy



Una base de datos relacional organiza la información en tablas conectadas por llaves.



CRUD se escribe en SQL puro: INSERT, SELECT, UPDATE y DELETE.



Normalizar evita duplicar datos y mantiene todo coherente al actualizar.



Las restricciones de integridad (NOT NULL, UNIQUE, FK, CHECK) impiden datos imposibles.



Las transacciones ACID garantizan que las operaciones críticas nunca queden a medias.



Los JOIN combinan tablas; INNER, LEFT, RIGHT y FULL OUTER cambian qué filas sin pareja se conservan.



Los índices aceleran las consultas, a cambio de algo de espacio y de velocidad de escritura.

Lo que ganamos al diseñarlo bien

Cada cosa que hemos visto hoy existe para conseguir uno de estos tres resultados.



La normalización evita que…

los datos se dupliquen y queden contradictorios entre tablas.



La integridad y ACID evitan que…

un fallo o un error a medio camino deje los datos rotos o incompletos.



Los índices evitan que…

las consultas se vuelvan lentas cuando la tabla crece a millones de filas.

04

¡Hora de jugar!

Comprobemos qué se ha quedado con un Kahoot



Kahoot: Bases de Datos Relacionales

Unas preguntas rápidas para repasar en vivo lo que hemos visto: tablas y llaves, normalización, integridad, ACID, SQL y qué hace un índice.



Entra en kahoot.it
y escribe el PIN del juego

¿Cómo se juega?



Un dispositivo por persona

Con el móvil o el portátil, sin necesidad de descargar nada: solo el navegador.



Preguntas con tiempo límite

Cada pregunta tiene 4 opciones y un contador. Cuanto antes aciertes, más puntos.



Marcador en vivo

Después de cada pregunta se ve la clasificación, para mantener el ritmo y la diversión.

Al terminar, repasamos juntos las preguntas que más hayan costado.



¿Preguntas?

¡Gracias!

Código y material en GitHub

github.com/arita-pereira/bases-de-datos-relacionales-sql